

Detalhes de Implementação — Etapa 3 do Compilador Saphira

Este guia explica detalhadamente o **que** foi feito, **como** foi feito e **por que** foi feito na Etapa 3 do compilador Saphira. Ele foi elaborado para que você entenda profundamente o funcionamento interno e consiga explicar o projeto com propriedade em apresentações.

1. O que foi feito (As Funcionalidades)

Expandimos o compilador para suportar um conjunto completo de estruturas de linguagens de programação modernas:

1. **Funções:** Definição com tipo de retorno (`int` , `float` , `char` , `bool` , `string` ou `void`), suporte a múltiplos parâmetros tipados, variáveis locais com escopo restrito e controle estrito de tipo de retorno (`return`).
 2. **Vetores e Matrizes (1D e 2D):** Declarações simples (ex: `int v[5];` , `int m[2][3];`) e inicializações automáticas com listas de valores (ex: `int v[5] = {10, 20, 30};` ou matrizes bidimensionais `int m[2][3] = {{1, 2, 3}, {4, 5, 6}};`).
 3. **Inicializações Inline e Múltiplas:** Declaração de múltiplas variáveis do mesmo tipo na mesma linha, com ou sem atribuição direta (ex: `int a = 5, b = 10, c;`).
 4. **Operadores Compostos e Unários:** Suporte a atribuições compostas (`+=` , `-=` , `*=` , `/=`) e operadores unários de incremento/decremento pré e pós-fixados (`a++` , `--b`), além de controle de sinal unitário (`+` e `-`).
 5. **Enumerações (Enums):** Definição de constantes enumeradas (ex: `enum Estado { OFF, ON };`) que mapeiam para constantes numéricas inteiras.
 6. **Slices (Fatiamento de Vetores):** Atribuição direta de pedaços de vetores (`dest = src[i:j];`) e exibição direta formatada de fatias via `write(src[i:j]);` .
 7. **Detecção de Erros Semânticos Robustos:** Todas as novas construções possuem validação rígida de tipos e escopos, imprimindo a linha exata onde ocorreu a incoerência.
-

2. Como foi feito (A Implementação Técnica)

Abaixo estão descritas as principais engrenagens implementadas nos componentes do compilador.

2.1. Tabela de Símbolos (`src/simbolos.h` e `src/simbolos.cpp`)

Para acomodar as novas estruturas, a classe `Simbolo` foi expandida. Agora ela rastreia:

- Se o símbolo é um vetor (`eh_vetor`) ou matriz (`eh_matriz`), salvando os limites das dimensões em `dim1` e `dim2` .
- Se o símbolo é uma função (`eh_funcao`), salvando o tipo de retorno em `tipo_retorno` e a assinatura de tipos dos parâmetros em um vetor `tipos_parametros` .
- Se o símbolo é uma constante de enum (`eh_enum_const`), mapeando seu valor inteiro associado.

2.2. Analisador Léxico (`src/lexico.l`)

- Adicionados padrões léxicos para operadores unários (`++` , `--`) e compostos (`+=` , `-=` , `*=` , `/=`), além das palavras-chave `void` , `return` e `enum` .
- Ativado o contador automático de linhas do Flex `%option yylineno` . Isso garante que qualquer erro sintático ou semântico indique a linha exata da ocorrência usando a variável global `yylineno` .

2.3. Analisador Sintático & Semântico (`src/sintatico.y`)

Aqui residem as soluções de engenharia mais refinadas:

A. Redirecionamento Dinâmico de Buffers para Funções

Em C, funções devem ser declaradas fora da função `main()` . Como a linguagem Saphira permite escrever funções no meio do código, utilizamos buffers de texto separados.

- Criamos `buffer_funcoes_global` (para guardar o código das funções) e ponteiros dinâmicos `buffer_atual` e `buffer_decls_atual` .
- Por padrão, os ponteiros apontam para os buffers globais da `main()` .
- Ao iniciar a compilação de uma função, redirecionamos esses ponteiros para buffers locais da função (`buffer_codigo_funcao` e `buffer_decls_funcao`).
- O gerador de temporários é reiniciado (`gerador.reiniciar()`) para que a função comece a usar temporários a partir de `t1` localmente.
- Ao final da redução da função, construímos o cabeçalho e corpo da função a partir dos buffers locais da função e salvamos tudo no buffer global de funções, restaurando os ponteiros dinâmicos para a `main()` .

B. Solução para Declarações Inline sem Conflitos LALR(1)

No Yacc/Bison padrão, a obtenção de tipos em regras aninhadas (como `int a = 5, b = 10;`) costuma ser feita lendo offsets da pilha (`$<int_val>0`). Porém, em listas recursivas, o

offset varia conforme o nível da recursão. A tentativa de contornar isso usando ações sintáticas intermediárias gera conflitos insolúveis de shift/reduce (deslocamento/redução).

- **Nossa Solução:** Adicionamos uma variável global no compilador chamada `tipo_declaracao_atual`. Sempre que o parser reduz o token de um tipo primitivo (como `int`, `float`, etc.), a regra `tipo` atualiza essa variável imediatamente.
- Dessa forma, qualquer item declarado dentro da regra `lista_declaracoes` acessa diretamente o valor de `tipo_declaracao_atual`, garantindo compatibilidade e eliminando conflitos de lookahead do parser.

C. Tradução de Enums e Slices

- **Enums:** Quando o parser encontra `enum Estado { OFF, ON };`, ele insere `OFF` com valor `0` e `ON` com valor `1` na Tabela de Símbolos como constantes globais inteiras. No C gerado, ele escreve a definição `enum Estado { OFF, ON };` no topo do arquivo.
- **Slices:** Um fatiamento como `dest = v[1:4]` é traduzido pelo compilador gerando um laço `for` compacto em C que itera de `0` até o tamanho da fatia (`4 - 1 = 3`), copiando ordenadamente os elementos:

```
t7 = 4 - 1;
for (int t6 = 0; t6 < t7; t6++) {
    dest[t6] = v[1 + t6];
}
```

3. Por que foi feito assim (Justificativas de Design)

1. Geração de C Nativos e Sem Bibliotecas Externas:

Queríamos que o código intermediário gerasse C 100% legível e compilável no GCC padrão sem exigir um arquivo runtime (`runtime.h` ou `stdbool.h`). Mapeamos enums Saphira para enums C, strings para `char[256]` tratadas com funções de `<string.h>` (`strcpy`, `strcat`), e booleanos diretamente para `int` (com `0` e `1`), mantendo a saída limpa e auto-suficiente.

2. Uso de Variável Global para Tipos nas Declarações:

Tentar usar herança de atributos na pilha do Bison em regras complexas como matrizes e listas inicializadas introduzia conflitos LALR(1) porque o Bison precisa decidir o tipo antes de ler o resto da linha. Utilizar a variável global `tipo_declaracao_atual` contornou essa limitação de lookahead mantendo a gramática extremamente limpa e livre de conflitos.

3. Escopo Isolado em Funções:

O isolamento de variáveis locais e a reinicialização de temporários garantem que variáveis

de funções não interfiram com variáveis da `main` ou de outras funções, mantendo a semântica de linguagens estruturadas de alto nível.

4. Guia Rápido para Apresentações (Oral Defense)

Se perguntado pelos avaliadores, você pode focar nesses três pilares:

- **Pergunta: Como a sua linguagem impede a mistura de tipos (ex: somar bool com int) se o código final é em C (que aceita isso)?**
 - **Resposta:** "Nossa verificação é feita em tempo de compilação pelo compilador Saphira. Se o compilador detectar tipos inválidos na análise semântica das expressões no `sintatico.y`, ele aborta a compilação imediatamente com `exit(1)` e exibe uma mensagem de erro semântico clara. O GCC só é chamado se o programa for semanticamente correto."
- **Pergunta: Como vocês geram as funções no topo do arquivo se elas podem ser definidas em qualquer ordem na linguagem Saphira?**
 - **Resposta:** "Utilizamos um buffer global específico para funções. Durante a compilação linear do Saphira, sempre que uma função é identificada, seu código correspondente é desviado para esse buffer. No final, o compilador imprime os cabeçalhos das enums, seguidos das funções compiladas e, por fim, a função `main()`. Isso garante que o código C resultante siga a estrutura sequencial correta exigida pela linguagem C."
- **Pergunta: O que acontece com as variáveis de temporários criadas em funções? Elas não colidem com a main?**
 - **Resposta:** "Não. Sempre que o compilador entra em uma função, chamamos o método `gerador.reiniciar()`. Isso limpa e reinicia o gerador de temporários locais daquela função. E como as variáveis locais de uma função pertencem ao seu escopo no C, elas ficam restritas àquele bloco de execução."